

TrustGuard

Data Quality Pipeline for Retail Data

Project Report

Built using Python, PySpark, SQL, Delta Lake, and Databricks

Field	Details
Domain	Data Engineering / Data Quality / Retail Analytics
Architecture	Medallion Architecture - Raw Layer -> Clean Layer -> Final Layer
Execution Platform	Databricks Free Edition with Serverless Compute
Project Folder Tool	VS Code for documentation, exported notebook, dataset, screenshots, and GitHub submission
Main Dataset	trustguard_retail_transactions.csv

This report explains the complete TrustGuard project in simple words. It also includes Databricks screenshots that show the pipeline output at each important step.

1. Project Overview

TrustGuard is an end-to-end data quality pipeline built for retail transaction data. The project takes raw and inconsistent data, checks it for common data quality issues, cleans it, separates invalid records, and creates final tables that can be used for business reporting and analysis.

The actual pipeline was executed on Databricks using PySpark, SQL, and Delta Lake. VS Code was used to maintain the final project folder, documentation, screenshots, exported notebook, dataset, and GitHub submission files.

2. Business Problem

Retail data often comes from different systems such as stores, online orders, customer platforms, payment systems, and delivery systems. Because of this, the data may contain missing values, duplicate records, inconsistent date formats, wrong total amounts, and different spellings for the same values.

If this data is used directly for reporting, revenue numbers, customer analysis, and product sales reports can become incorrect. TrustGuard solves this by applying validation, cleaning, rejected record handling, and reporting in a structured pipeline.

3. Architecture Used

Layer	Purpose	Main Tables
Raw Layer	Stores original data with metadata. No cleaning is applied here.	raw_transactions
Clean Layer	Applies validation, standardization, deduplication, and separates invalid records.	clean_transactions, clean_customers, rejected_records, dq_report
Final Layer	Stores analysis-ready tables and reports for business use.	final_transactions, customer_summary, city_sales_report, payment_method_report, product_category_report, anomaly_log, pipeline_log

Simple flow: Raw CSV Dataset -> Raw Delta Table -> Data Quality Checks -> Cleaned Data + Rejected Records -> Final Tables -> SQL Reports and Logs.

4. Data Quality Checks and Cleaning Rules

Check / Rule	What it does
Completeness Check	Finds missing values in important fields like transaction_id, customer_id, product_category, payment_method, quantity, unit_price, total_amount, and transaction_date.
Duplicate Check	Finds repeated transaction_id values and keeps only one valid record in the clean table.
Date Standardization	Converts mixed date formats into a consistent date format wherever possible.
Text Standardization	Standardizes values like payment method, gender, city, and active status.
Range Check	Checks that quantity and price values are valid and greater than zero.
Amount Validation	Checks whether total_amount matches quantity multiplied by unit_price.
Rejected Records	Stores records that fail critical checks with a clear rejection reason.
Anomaly Detection	Flags unusual transactions such as very high quantity or very high transaction amount.

The pipeline does not silently delete invalid records. Instead, it sends them to the rejected_records table with a reason. This makes the pipeline easier to audit, explain, and debug.

5. Important Output Tables

Table	Purpose
raw_transactions	Original uploaded data with extra metadata columns such as run_id, load_timestamp, and source_file_name.
dq_report	Shows data quality check results with passed records, failed records, and quality score.
rejected_records	Stores invalid records with clear rejection reasons.
clean_transactions	Stores cleaned transaction data after validation and deduplication.
clean_customers	Stores unique customer records derived from the cleaned transactions.
final_transactions	Main final analysis-ready transaction table.

Table	Purpose
customer_summary	Customer-level spend and order summary.
city_sales_report	City-wise and month-wise sales report.
anomaly_log	Records unusual transactions for review.
pipeline_log	Tracks pipeline run status and record counts.

6. Dataset Uploaded in Databricks

This screenshot shows the raw retail transaction dataset uploaded inside Databricks Catalog. The sample data confirms that the CSV was read correctly with columns such as transaction_id, customer_id, full_name, email, phone, gender, and date_of_birth.

databricks

Free Edition

+

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Catalog

Serverless Starter Ware...

Serverless

2XS

Catalog

Type to search

My organization

workspace

default

Tables (1)

trustguard_retail_transac...

information_schema

system

Shares received

samples

Catalog Explorer > workspace > default >

trustguard_retail_transactions

Open in a dashboard

Share

Create

Overview

Sample Data

Details

Permissions

Policies

History

Lineage

Insights

Quality

Ask your question about the sample data...

Ask

Reset

What is the most popular product category?

Which stores have the highest total sales?

How many transactions are made by loyalty tier?

Sample

	transaction_id	customer_id	full_name	email	phone	gender	date_of_birth
1	TXN-00000001	CUST-00951	Karan Sinha	karan.sinha340@gmail.com	+91-64788055...	M	05-17-1993
2	TXN-00000002	CUST-00072	Sahil Patel	sahil.patel859@rediffmail.com	7178884276	MALE	11-25-1993
3	TXN-00000003	CUST-01036	Harsh Sharma	harsh.sharma622@outlook.com	79596-10945	M	03-12-1992
4	TXN-00000004	CUST-01382	Sakshi Sharma	sakshi.sharma56@yahoo.com	76711-28657	FEMALE	02-25-2000
5	TXN-00000005	CUST-02194	Priya Joshi	priya.joshi793@yahoo.com	99930-16643	female	06/04/1985
6	TXN-00000006	CUST-00563	Isha Iyer	NULL	6779355497	FEMALE	10/01/1997
7	TXN-00000007	CUST-02027	Karan Joshi	karan.joshi135@rediffmail.com	+91-61090848...	Male	1994-07-04
8	TXN-00000008	CUST-00559	Varun Das	varun.das931@rediffmail.com	7455984805	M	11-15-2005
9	TXN-00000009	CUST-00044	Kavya Verma	kavya.verma22@rediffmail.com	95318-84396	F	07-12-1978
10	TXN-00000010	CUST-01634	Kavya Gupta	kavya.gupta609@gmail.com	+91-99772838...	female	15/04/2005
11	TXN-00000011	CUST-00424	Arjun Mehta	arjun.mehta892@gmail.com	7472520899	MALE	2005-03-20
12	TXN-00000012	CUST-00144	Nisha Sharma	nisha.sharma677@yahoo.com	8298461504	FEMALE	2004-03-14
13	TXN-00000013	CUST-00049	Vikram Patel	vikram.patel748@yahoo.com	7355268102	MALE	09-25-1998
14	TXN-00000014	CUST-00000	Karan Sinha	karan.sinha340@gmail.com	+91-64788055...	M	05-17-1993

7. Raw Table Loaded in Notebook

This output confirms that the uploaded Databricks table was successfully loaded into a PySpark DataFrame. It shows 10,250 rows and 24 columns, so the notebook can now use the dataset for the pipeline.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python

Last edit was 2 minutes ago

Run all Serverless Schedule Share

2 minutes ago (3s) 6 Python

```
raw_df = spark.table("workspace.default.trustguard_retail_transactions")

print("Raw row count:", raw_df.count())
print("Raw column count:", len(raw_df.columns))

display(raw_df.limit(10))
```

See performance (2) Checked for improvements

raw_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 22 more fields]

Raw row count: 10250

Raw column count: 24

Table

	transaction_id	customer_id	full_name	email	phone	gender	date_of_birth	city	state
1	TXN-00000001	CUST-00951	Karan Sinha	karan.sinha340@gmail.com	+91-64788055...	M	05-17-1993	Bangalore	Karnataka
2	TXN-00000002	CUST-00072	Sahil Patel	sahil.patel859@rediffmail.com	7178884276	MALE	11-25-1993	Indore	Madhya Prade
3	TXN-00000003	CUST-01036	Harsh Sharma	harsh.sharma622@outlook.co...	79596-10945	M	03-12-1992	Hyderabad	Telangana
4	TXN-00000004	CUST-01382	Sakshi Sharma	sakshi.sharma56@yahoo.com	76711-28657	FEMALE	02-25-2000	Jaipur	Rajasthan
5	TXN-00000005	CUST-02194	Priya Joshi	priya.joshi793@yahoo.com	99930-16643	female	06/04/1985	Bangalore	Karnataka
6	TXN-00000006	CUST-00563	Isha Iyer	NULL	6779355497	FEMALE	10/01/1997	Bangalore	Karnataka
7	TXN-00000007	CUST-02027	Karan Joshi	karan.joshi135@rediffmail.com	+91-61090848...	Male	1994-07-04	Bhopal	Madhya Prade
8	TXN-00000008	CUST-00559	Varun Das	varun.das931@rediffmail.com	7455984805	M	11-15-2005	Kolkata	West Bengal
9	TXN-00000009	CUST-00044	Kavya Verma	kavya.verma22@rediffmail.com	95318-84396	F	07-12-1978	Jaipur	Rajasthan
10									

10 rows | 3.299s runtime

Refreshed 2 minutes ago

8. Raw Layer Delta Table

This screenshot shows the Raw Layer table created successfully. The raw data was stored with metadata columns such as run_id, load_timestamp, and source_file_name. No cleaning was applied in this layer.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python Last edit was now

Just now (12s)

8

Python

```
raw_df
.withColumn("run_id", F.lit(run_id))
.withColumn("load_timestamp", F.current_timestamp())
.withColumn("source_file_name", F.lit("trustguard_retail_transactions.csv"))
)

raw_with_metadata_df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("trustguard_db.raw_transactions")

print("Raw Layer table created successfully.")
print("Raw records:", raw_with_metadata_df.count())

display(spark.table("trustguard_db.raw_transactions").limit(10))
```

See performance (3) ✓ Checked for improvements

raw_with_metadata_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 25 more fields]

Raw Layer table created successfully.

Raw records: 10250

Table

+

	transaction_id	customer_id	full_name	email	phone	gender	date_of_birth	city	state	registration_date
1	TXN-00000001	CUST-00951	Karan Sinha	karan.sinha340@gmail.com	+91-64788055...	M	05-17-1993	Bangalore	Karnataka	10/20/22
2	TXN-00000002	CUST-00072	Sahil Patel	sahilpatel859@rediffmail.com	7178884276	MALE	11-25-1993	Indore	Madhya Prade...	2023-09-24
3	TXN-00000003	CUST-01036	Harsh Sharma	harsh.sharma622@outlook.co...	79596-10945	M	03-12-1992	Hyderabad	Telangana	13-07-2023
4	TXN-00000004	CUST-01382	Sakshi Sharma	sakshi.sharma56@yahoo.com	76711-28657	FEMALE	02-25-2000	Jaipur	Rajasthan	2023-04-07
5	TXN-00000005	CUST-02194	Priya Joshi	priya.joshi793@yahoo.com	99930-16643	female	06/04/1985	Bangalore	Karnataka	01-09-2021
6	TXN-00000006	CUST-00563	Isha Iyer	NULL	6779355497	FEMALE	10/01/1997	Bangalore	Karnataka	12-04-2021
7	TXN-00000007	CUST-02027	Karan Joshi	karan.joshi135@rediffmail.com	+91-61090848...	Male	1994-07-04	Bhopal	Madhya Prade...	21-03-2021
8	TXN-00000008	CUST-00559	Varun Das	varun.das931@rediffmail.com	7455984805	M	11-15-2005	Kolkata	West Bengal	10/12/21
9	TXN-00000009	CUST-00044	Kavya Verma	kavya.verma22@rediffmail.com	95318-84396	F	07-12-1978	Jaipur	Rajasthan	02-07-2023
10										

TrustGuard Project Report

Page 7

9. Initial Data Quality Report

This output shows the data quality report for the raw data. It counts failed records for missing values and duplicate transaction IDs. This helps identify how dirty the incoming data is before cleaning.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python

Run all Serverless Schedule Share

Just now (14s) 9 Python

```
StructField("total_records", IntegerType(), True),
StructField("passed_records", IntegerType(), True),
StructField("failed_records", IntegerType(), True),
StructField("dq_score", DoubleType(), True),
StructField("run_timestamp", StringType(), True)
})

dq_report_df = spark.createDataFrame(dq_rows, dq_schema)

dq_report_df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("trustguard_db.dq_report")

display(dq_report_df)
```

See performance (12)

raw_transactions: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 25 more fields]

dq_report_df: pyspark.sql.connect.dataframe.DataFrame = [run_id: string, table_name: string ... 6 more fields]

Table

	run_id	table_name	check_name	total_records	passed_records	failed_records	dq_score	run_timestamp
1	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_transaction_id	10250	10225	25	99.76	2026-07-11 15:22:55
2	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_customer_id	10250	10228	22	99.79	2026-07-11 15:22:55
3	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_product_category	10250	9810	440	95.71	2026-07-11 15:22:55
4	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_payment_method	10250	10155	95	99.07	2026-07-11 15:22:55
5	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_quantity	10250	10250	0	100	2026-07-11 15:22:55
6	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_unit_price	10250	10250	0	100	2026-07-11 15:22:55
7	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_total_amount	10250	10250	0	100	2026-07-11 15:22:55
8	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_transaction_date	10250	10250	0	100	2026-07-11 15:22:55
9	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	duplicate_transaction_id_che...	10250	10000	250	97.56	2026-07-11 15:22:55

9 rows | 13.546s runtime

Refreshed now

10. Rejected Records

This table contains records that failed critical checks. The rejection_reason column clearly explains why each record was rejected, such as total amount mismatch, missing customer_id, or non-positive quantity.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python

1 minute ago (8s)

12

Python

Run all Serverless Schedule Share

```
print("Rejected records:", rejected_records_df.count())
display(rejected_records_df.select("transaction_id", "customer_id", "rejection_reason").limit(20))
```

See performance (3) ✓ Checked for improvements

rejected_records_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 39 more fields]

validated_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 38 more fields]

Rejected records: 338

Table

	transaction_id	customer_id	rejection_reason
1	TXN-00000033	CUST-00668	Total amount mismatch
2	TXN-00000065	CUST-02170	Total amount mismatch
3	TXN-00000076	null	Missing customer_id
4	TXN-00000126	CUST-01945	Non-positive quantity
5	TXN-00000146	CUST-01699	Non-positive quantity
6	TXN-00000162	CUST-01460	Non-positive quantity
7	TXN-00000204	CUST-00178	Non-positive quantity
8	TXN-00000281	CUST-01919	Non-positive quantity
9	TXN-00000386	CUST-01577	Total amount mismatch
10	TXN-00000391	CUST-01023	Total amount mismatch
11	TXN-00000413	CUST-01523	Total amount mismatch
12	TXN-00000421	CUST-01475	Non-positive quantity
13	TXN-00000471	CUST-01301	Non-positive quantity
14	TXN-00000498	CUST-01114	Non-positive quantity
15	TXN-00000503	CUST-01091	Total amount mismatch

20 rows | 8.155s runtime

Refreshed 1 minute ago

11. Clean Transactions Table

This screenshot shows the cleaned transactions after applying validation, standardization, deduplication, and type casting. The clean record count is lower than the raw count because invalid and duplicate records were handled.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python Last edit was 1 minute ago

1 minute ago (3s)

13

Python

```
)

clean_transactions_df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("trustguard_db.clean_transactions")

print("Clean transactions:", clean_transactions_df.count())
display(clean_transactions_df.limit(10))
```

See performance (3) ✓ Checked for improvements

clean_transactions_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 25 more fields]

deduped_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 38 more fields]

valid_clean_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 38 more fields]

Clean transactions: 9668

Table

transaction_id customer_id full_name email phone gender date_of_birth city state registration_date

1 TXN-00000001 CUST-00951 Karan Sinha karan.sinha340@gmail.com 916478805564 Male 1993-05-17 Bangalore Karnataka null

2 TXN-00000002 CUST-00072 Sahil Patel sahil.patel859@rediffmail.com 7178884276 Male 1993-11-25 Indore Madhya Prade... 2023-09-24

3 TXN-00000003 CUST-01036 Harsh Sharma harsh.sharma622@outlook.co... 7959610945 Male 1992-12-03 Hyderabad Telangana 2023-07-13

4 TXN-00000004 CUST-01382 Sakshi Sharma sakshi.sharma56@yahoo.com 7671128657 Female 2000-02-25 Jaipur Rajasthan 2023-04-07

5 TXN-00000005 CUST-02194 Priya Joshi priya.joshi793@yahoo.com 9993016643 Female 1985-04-06 Bangalore Karnataka 2021-09-01

6 TXN-00000006 CUST-00563 Isha Iyer null 6779355497 Female 1997-01-10 Bangalore Karnataka 2021-04-12

7 TXN-00000007 CUST-02027 Karan Joshi karan.joshi135@rediffmail.com 916109084830 Male 1994-07-04 Bhopal Madhya Prade... 2021-03-21

8 TXN-00000008 CUST-00559 Varun Das varun.das931@rediffmail.com 7455984805 Male 2005-11-15 Kolkata West Bengal null

9 TXN-00000009 CUST-00044 Kavya Verma kavya.verma22@rediffmail.com 9531884396 Female 1978-12-07 Jaipur Rajasthan 2023-07-02

10 rows | 9.012s runtime Refreshed 1 minute ago

12. Final Transactions Table

This output shows the final analysis-ready transaction table. It contains clean transaction data joined with customer details and is used for business reporting.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python

Just now (13s) 15

Python

Run all Serverless Schedule Share

```
F.col("t.run_id")
)

final_transactions_df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("trustguard_db.final_transactions")

print("Final transactions:", final_transactions_df.count())
display(final_transactions_df.limit(10))
```

See performance (3) Checked for improvements

final_transactions_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 20 more fields]

Final transactions: 9668

Table

	transaction_id	customer_id	full_name	email	phone	gender	city	state	loyalty_tier	is_active	
1	TXN-00000001	CUST-00951	Karan Sinha	karan.sinha340@gmail.com	916478805564	Male	Bangalore	Karnataka	Bronze	Active	202
2	TXN-00000002	CUST-00072	Sahil Patel	sahil.patel859@rediffmail.com	7178884276	Male	Indore	Madhya Prade...	Platinum	Active	202
3	TXN-00000003	CUST-01036	Harsh Sharma	harsh.sharma622@outlook.co...	7959610945	Male	Hyderabad	Telangana	Platinum	Unknown	202
4	TXN-00000004	CUST-01382	Sakshi Sharma	sakshi.sharma56@yahoo.com	7671128657	Female	Jaipur	Rajasthan	Silver	Active	202
5	TXN-00000005	CUST-02194	Priya Joshi	priya.joshi793@yahoo.com	9993016643	Female	Bangalore	Karnataka	Silver	Unknown	202
6	TXN-00000006	CUST-00563	Isha Iyer	null	6779355497	Female	Bangalore	Karnataka	Platinum	Active	202
7	TXN-00000007	CUST-02027	Karan Joshi	karan.joshi135@rediffmail.com	916109084830	Male	Bhopal	Madhya Prade...	Silver	Active	202
8	TXN-00000008	CUST-00559	Varun Das	varun.das931@rediffmail.com	7455984805	Male	Kolkata	West Bengal	Silver	Active	202
9	TXN-00000009	CUST-00044	Kavya Verma	kavya.verma22@rediffmail.com	9531884396	Female	Jaipur	Rajasthan	Bronze	Unknown	202
10											

10 rows | 13.466s runtime Refreshed now

13. City Sales Report

This report shows city-wise and month-wise sales performance. It includes total orders, total revenue, and average order value, which are useful for business analysis.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Catalog

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python Last edit was now

Run all

Serverless

Schedule

Share

Just now (11s)

17

Python

```
        )
        .orderBy(F.col("total_revenue").desc())
    )

    city_sales_report_df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("trustguard_db.city_sales_report")

    display(city_sales_report_df.limit(20))
```

See performance (2)

Checked for improvements

city_sales_report_df: pyspark.sql.connect.dataframe.DataFrame = [city: string, state: string ... 4 more fields]

Table

city

state

sales_month

total_orders

total_revenue

average_order_value

1	Indore	Madhya Prade...	2024-03	164	550331.51	3355.68
2	Lucknow	Uttar Pradesh	2024-03	133	500057.3	3759.83
3	Delhi	Delhi	2024-03	126	460482.45	3654.62
4	Pune	Maharashtra	2024-01	108	437468.38	4050.63
5	Chandigarh	Chandigarh	2024-05	126	435832.34	3458.99
6	Lucknow	Uttar Pradesh	2024-05	121	426717.44	3526.59
7	Hyderabad	Telangana	2024-05	115	414437.23	3603.8
8	Chennai	Tamil Nadu	2024-05	128	406095.5	3172.62
9	Surat	Gujarat	2024-04	114	405231.78	3554.66
10	Hyderabad	Telangana	2024-03	111	395934.95	3566.98
11	Lucknow	Uttar Pradesh	2024-04	118	391593	3318.58
12	Delhi	Delhi	2024-06	106	388886.73	3668.74
13	Surat	Gujarat	2024-03	134	388770.54	2901.27
14	Indore	Madhya Prade...	2024-06	126	384173.71	3049
15	Chennai	Tamil Nadu	2024-06	121	383787.63	3171.8

20 rows

10.807s runtime

Refreshed now

14. Anomaly Log

This screenshot shows unusual transactions flagged by the anomaly detection step. These records may have very high quantity or transaction amount and can be reviewed separately.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python

Last edit was 3 minutes ago

Run all Serverless Schedule Share

1 minute ago (13s) 20 Python

```
)  
  
anomaly_df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("trustguard_db.anomaly_log")  
  
print("Anomaly records:", anomaly_df.count())  
display(anomaly_df.limit(20))
```

See performance (4)

Checked for improvements

anomaly_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 22 more fields]

Anomaly records: 240

Table

	transaction_id	customer_id	full_name	email	phone	gender	city	state	loyalty_tier	is_active	
1	TXN-0000010	CUST-01634	Kavya Gupta	kavya.gupta609@gmail.com	919977283871	Female	Kolkata	West Bengal	null	Active	202
2	TXN-0000102	CUST-00308	Nisha Chopra	nisha.chopra395@yahoo.com	8217799930	Female	Delhi	Delhi	Bronze	Active	202
3	TXN-0000117	CUST-01053	Nikhil Reddy	nikhil.reddy78@yahoo.com	8500188574	Male	Jaipur	Rajasthan	Bronze	Active	202
4	TXN-0000122	CUST-01721	Aarav Khan	aarav.khan532@rediffmail.com	6720043822	Male	Lucknow	Uttar Pradesh	Bronze	Active	202
5	TXN-0000224	CUST-01218	Meera Khan	meera.khan366@rediffmail.co...	916699175664	Female	Jaipur	Rajasthan	Bronze	Active	202
6	TXN-0000238	CUST-01614	Rahul Khan	rahul.khan774@gmail.com	916256884011	Male	Pune	Maharashtra	Platinum	Active	202
7	TXN-0000266	CUST-00937	Nisha Singh	nisha.singh643@yahoo.com	919621683021	Female	Surat	Gujarat	Bronze	Active	202
8	TXN-0000325	CUST-01999	Priya Chopra	priya.chopra236@yahoo.com	8823975269	Female	Lucknow	Uttar Pradesh	null	Active	202
9	TXN-0000327	CUST-02146	Arjun Reddy	arjun.reddy319@rediffmail.com	6896041477	Male	Bhopal	Madhya Prade...	Platinum	Unknown	202
10	TXN-0000365	CUST-01090	Karan Chopra	karan.chopra274@outlook.com	918712377182	Male	Ahmedabad	Gujarat	null	Active	202
11	TXN-0000379	CUST-00799	Nisha Nair	null	916134458673	Female	Jaipur	Rajasthan	Silver	Active	202
12	TXN-0000405	CUST-02152	Karan Reddy	karan.reddy630@yahoo.com	6543077963	Male	Indore	Madhya Prade...	Gold	Active	202
13	TXN-0000479	CUST-01834	Simran Chopra	null	916625224134	Female	Mumbai	Maharashtra	Silver	Active	202
14	TXN-0000551	CUST-01731	Meera Patel	meera.patel494@outlook.com	916106256298	Female	Hyderabad	Telangana	Bronze	Active	202

20 rows | 13.356s runtime

Refreshed now

15. Pipeline Log

The pipeline log tracks each major pipeline step with record counts and status. It helps confirm that the pipeline ran successfully and makes the workflow easier to monitor.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python

Last edit was 3 minutes ago

Run all Serverless Schedule Share

1 minute ago (13s) 20 Python

```
)  
  
anomaly_df.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("trustguard_db.anomaly_log")  
  
print("Anomaly records:", anomaly_df.count())  
display(anomaly_df.limit(20))
```

See performance (4)

Checked for improvements

anomaly_df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 22 more fields]

Anomaly records: 240

Table

	transaction_id	customer_id	full_name	email	phone	gender	city	state	loyalty_tier	is_active	
1	TXN-0000010	CUST-01634	Kavya Gupta	kavya.gupta609@gmail.com	919977283871	Female	Kolkata	West Bengal	null	Active	202
2	TXN-00000102	CUST-00308	Nisha Chopra	nisha.chopra395@yahoo.com	8217799930	Female	Delhi	Delhi	Bronze	Active	202
3	TXN-00000117	CUST-01053	Nikhil Reddy	nikhil.reddy78@yahoo.com	8500188574	Male	Jaipur	Rajasthan	Bronze	Active	202
4	TXN-00000122	CUST-01721	Aarav Khan	aarav.khan532@rediffmail.com	6720043822	Male	Lucknow	Uttar Pradesh	Bronze	Active	202
5	TXN-00000224	CUST-01218	Meera Khan	meera.khan366@rediffmail.co...	916699175664	Female	Jaipur	Rajasthan	Bronze	Active	202
6	TXN-00000238	CUST-01614	Rahul Khan	rahul.khan774@gmail.com	916256884011	Male	Pune	Maharashtra	Platinum	Active	202
7	TXN-00000266	CUST-00937	Nisha Singh	nisha.singh643@yahoo.com	919621683021	Female	Surat	Gujarat	Bronze	Active	202
8	TXN-00000325	CUST-01999	Priya Chopra	priya.chopra236@yahoo.com	8823975269	Female	Lucknow	Uttar Pradesh	null	Active	202
9	TXN-00000327	CUST-02146	Arjun Reddy	arjun.reddy319@rediffmail.com	6896041477	Male	Bhopal	Madhya Prade...	Platinum	Unknown	202
10	TXN-00000365	CUST-01090	Karan Chopra	karan.chopra274@outlook.com	918712377182	Male	Ahmedabad	Gujarat	null	Active	202
11	TXN-00000379	CUST-00799	Nisha Nair	null	916134458673	Female	Jaipur	Rajasthan	Silver	Active	202
12	TXN-00000405	CUST-02152	Karan Reddy	karan.reddy630@yahoo.com	6543077963	Male	Indore	Madhya Prade...	Gold	Active	202
13	TXN-00000479	CUST-01834	Simran Chopra	null	916625224134	Female	Mumbai	Maharashtra	Silver	Active	202
14	TXN-00000551	CUST-01731	Meera Patel	meera.patel494@outlook.com	916106256298	Female	Hyderabad	Telangana	Bronze	Active	202
15											

20 rows | 13.356s runtime

Refreshed now

16. Tables Created in Databricks

This screenshot shows all tables created under the trustguard_db schema. It confirms that the Raw, Clean, Final, DQ, rejected, anomaly, and pipeline log tables were created successfully.

The screenshot displays the Databricks SQL interface. The top navigation bar includes the Databricks logo, a search bar, and user information. The left sidebar shows the workspace structure with options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, and Marketplace. The main area shows a SQL query cell with the command `SHOW TABLES IN trustguard_db;`. Below the query, a table lists the results of the query.

	database	tableName	isTemporary
1	trustguard_db	anomaly_log	false
2	trustguard_db	city_sales_report	false
3	trustguard_db	clean_customers	false
4	trustguard_db	clean_transactions	false
5	trustguard_db	customer_summary	false
6	trustguard_db	dq_report	false
7	trustguard_db	final_transactions	false
8	trustguard_db	payment_method_rep...	false
9	trustguard_db	pipeline_log	false
10	trustguard_db	product_category_rep...	false
11	trustguard_db	raw_transactions	false
12	trustguard_db	rejected_records	false

Below the table, a message states: "This result is stored as _sqlidf and can be used in other Python and SQL cells." The interface also shows a "Run all" button, a "Serverless" dropdown, and a "Schedule" button. The bottom right corner has a "Send feedback" link.

17. SQL Report - City Revenue

This SQL output shows total revenue and total orders by city and state. It proves that the final clean table can be queried for business reports.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

databricks
Free Edition

Search data, notebooks, recents, and more...
CTRL + P

Verify identityworkspace

TrustGuard_Databricks_Pipeline x

File Edit View Run Help Python ☆ Last edit was 1 minute ago

Run all Serverless Schedule Share

1 minute ago (8)

23

SQL

```
city,  
state,  
ROUND(SUM(total_amount), 2) AS total_revenue,  
COUNT(DISTINCT transaction_id) AS total_orders  
FROM trustguard_db.final_transactions  
GROUP BY city, state  
ORDER BY total_revenue DESC;
```

> See performance (1)
✓ Checked for improvements

> _sqldf: pyspark.sql.connect.dataframe.DataFrame = [city: string, state: string ... 2 more fields]

Table +

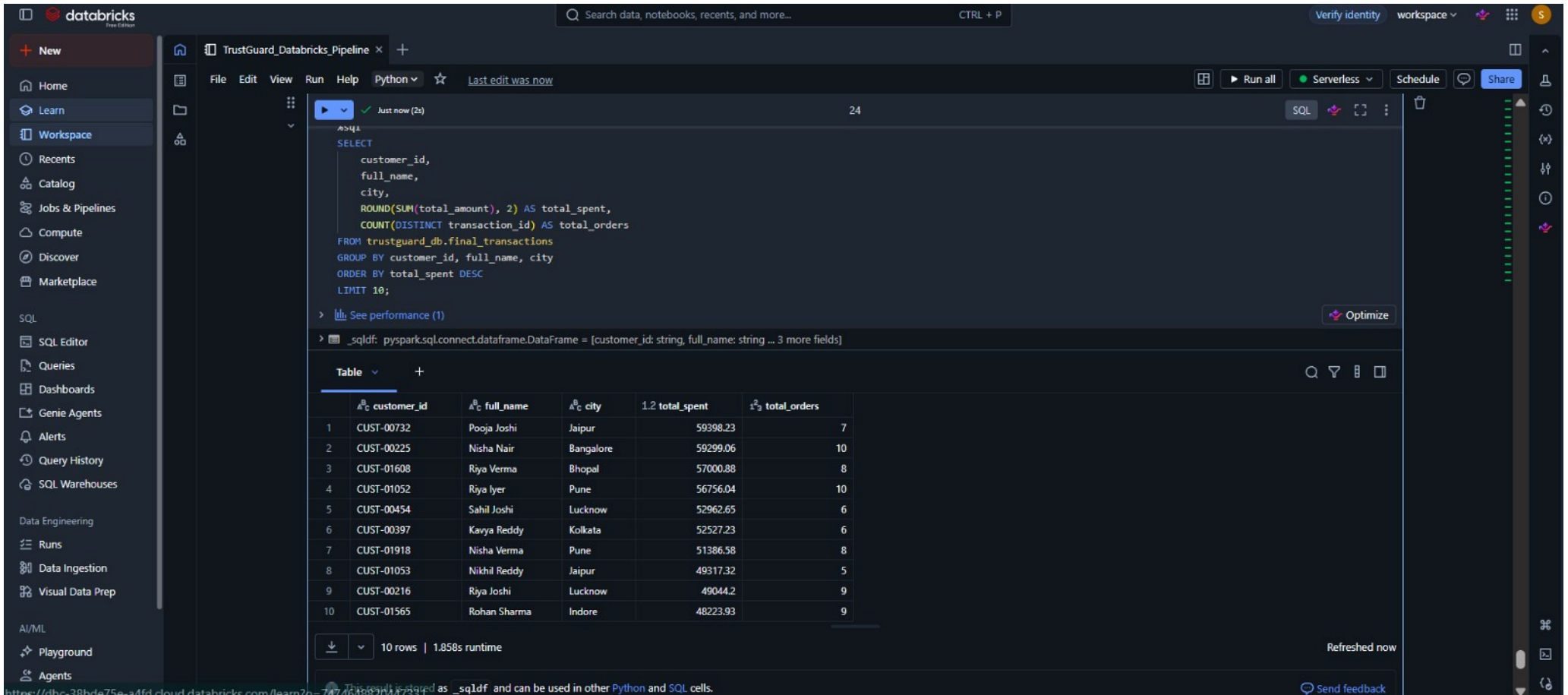
	city	state	total_revenue	total_orders
1	Indore	Madhya Prade...	2374908.99	738
2	Lucknow	Uttar Pradesh	2278559.31	663
3	Delhi	Delhi	2152365.25	638
4	Chandigarh	Chandigarh	2102485.34	682
5	Surat	Gujarat	2073856.62	695
6	Mumbai	Maharashtra	2071924	670
7	Chennai	Tamil Nadu	2065142.96	690
8	Hyderabad	Telangana	2046898.78	642
9	Kolkata	West Bengal	1942040.94	638
10	Pune	Maharashtra	1894530.82	569
11	Jaipur	Rajasthan	1876233.8	574
12	Bangalore	Karnataka	1838054.13	661
13	Bhopal	Madhya Prade...	1789780.19	599
14	Patna	Bihar	1785448.75	648
15	Ahmedabad	Gujarat	1744016.16	559

↓ 16 rows | 8.230s runtime

Refreshed 1 minute ago

18. SQL Report - Top Customers

This report shows the top 10 customers by total spend. It uses the final_transactions table and helps identify high-value customers.



The screenshot displays the Databricks SQL interface. The left sidebar contains navigation options: Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, and Agents. The main workspace shows a SQL query executed 'Just now (2s)' with a status of 'Success'. The query is as follows:

```
SELECT
  customer_id,
  full_name,
  city,
  ROUND(SUM(total_amount), 2) AS total_spent,
  COUNT(DISTINCT transaction_id) AS total_orders
FROM trustguard_db.final_transactions
GROUP BY customer_id, full_name, city
ORDER BY total_spent DESC
LIMIT 10;
```

Below the query, the results are displayed in a table format. The table has 6 columns: customer_id, full_name, city, total_spent, and total_orders. The results show the top 10 customers by total spend.

	customer_id	full_name	city	total_spent	total_orders
1	CUST-00732	Pooja Joshi	Jaipur	59398.23	7
2	CUST-00225	Nisha Nair	Bangalore	59299.06	10
3	CUST-01608	Riya Verma	Bhopal	57000.88	8
4	CUST-01052	Riya Iyer	Pune	56756.04	10
5	CUST-00454	Sahil Joshi	Lucknow	52962.65	6
6	CUST-00397	Kavya Reddy	Kolkata	52527.23	6
7	CUST-01918	Nisha Verma	Pune	51386.58	8
8	CUST-01053	Nikhil Reddy	Jaipur	49317.32	5
9	CUST-00216	Riya Joshi	Lucknow	49044.2	9
10	CUST-01565	Rohan Sharma	Indore	48223.93	9

The interface also shows a 'See performance (1)' link and an 'Optimize' button. At the bottom, it indicates '10 rows | 1.858s runtime' and 'Refreshed now'.

19. SQL Report - Rejected Records Summary

This SQL output groups rejected records by rejection reason. It helps understand the most common data quality problems in the raw dataset.

The screenshot displays the Databricks SQL interface. The top navigation bar includes the Databricks logo, a search bar, and user controls. The left sidebar lists various workspace components like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie Agents, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, Data Ingestion, Visual Data Prep, AI/ML, Playground, and Agents. The main workspace area shows a notebook titled 'TrustGuard_Databricks_Pipeline'. The active cell contains an SQL query that selects rejection reasons and counts the total records for each reason, ordered by total records in descending order. The query results are displayed in a table with 5 rows. Below the table, it indicates '5 rows | 2.236s runtime' and 'Refreshed 1 hour ago'. A message states that the result is stored as '_sqlidf' and can be used in other Python and SQL cells. The bottom of the interface shows another SQL cell with a query to select all records from 'trustguard_db.dq_report' ordered by failed records in descending order.

```
%sql
SELECT
  rejection_reason,
  COUNT(*) AS total_records
FROM trustguard_db.rejected_records
GROUP BY rejection_reason
ORDER BY total_records DESC;
```

	rejection_reason	total_records
1	Non-positive quantity	165
2	Total amount mismatch	124
3	Missing transaction_id	25
4	Missing customer_id	22
5	Non-positive quantity; Total amount mismatch	2

5 rows | 2.236s runtime

Refreshed 1 hour ago

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

```
%sql
SELECT *
FROM trustguard_db.dq_report
ORDER BY failed_records DESC;
```

20. SQL Report - Data Quality Summary

This output shows the data quality checks ordered by failed records. It gives a clear view of which checks had the highest number of failures.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

TrustGuard_Databricks_Pipeline

File Edit View Run Help Python Last edit was 1 hour ago

09:58 PM (2s) 26 SQL

```
%sql
SELECT *
FROM trustguard_db.dq_report
ORDER BY failed_records DESC;
```

See performance (1) ✓ Checked for improvements

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [run_id: string, table_name: string ... 6 more fields]

Table

	run_id	table_name	check_name	total_records	passed_records	failed_records
1	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_product_category	10250	9810	440
2	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	duplicate_transaction_id_che...	10250	10000	250
3	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_payment_method	10250	10155	95
4	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_transaction_id	10250	10225	25
5	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_customer_id	10250	10228	22
6	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_quantity	10250	10250	0
7	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_unit_price	10250	10250	0
8	9c3a1eb1-248a-4654-a0d6-b7331ee07f...	raw_transactions	null_check_total_amount	10250	10250	0
9	...	...	...	...	...	...

9 rows | 2.489s runtime

Refreshed 1 hour ago

TrustGuard Project Report

Page 19

